



# 产品进化智能体

基于 LangGraph 的多智能体产品需求决策系统

把分散的用户反馈与 GitHub Issue，变成有证据、有出口、可追溯的产品决策

LangGraph

GLM-5.1

Pydantic v2

React + Vite

14 节点 · 7 Agent

# 产品团队每天都在做、却很难做好的事

反馈在涨，决策的质量却没跟上 —— 三个反复出现的困境：

## 01

### 淹没

情绪宣泄、重复提交、用错功能混在一起，成百上千条反馈靠人工根本筛不过来，真问题被噪声淹没。

## 02

### 拍脑袋

优先级靠感觉和嗓门决定，缺乏证据支撑。做完了说不清「为什么做这个」，无法向团队和管理层交代。

## 03

### 断链

需求、竞品、技术可行性、代码影响面各算各的。最终 PRD 无法回溯到底依据了哪条反馈，结论不可追溯。

# 不做更聪明的「分类器」，做一条有纪律的「需求漏斗」

## 1

### 漏斗必须有出口

不是所有反馈都转需求，不是所有需求都值得做。过滤 · 查重 · 门禁 · 分流，每一级都有「淘汰 / 分流」记录，并在报告里展示出来。

27 → 5 → 3 → 1 → P0

## 2

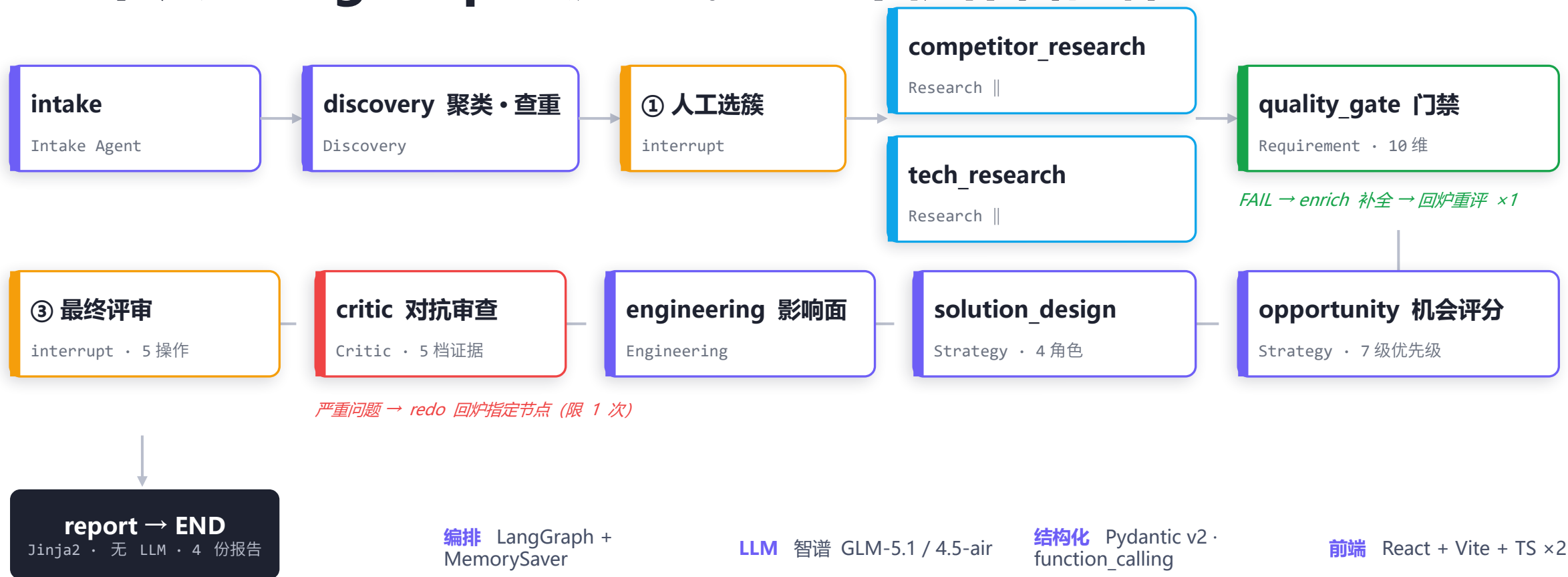
### 所有结论须证据闭包

每个判断都引用真实上游证据 id，节点输出后由代码层校验合法性；LLM 编造的引用被直接剔除并交对抗式审查。

evidence\_refs 全链校验

系统只输出建议文档，永不产出代码改动（无 Patch / 无 PR）——人工介入分级，执行权始终在手里。

# 14 节点 LangGraph 流水线 • 7 个协作智能体



# 每一级都有出口：淘汰与分流都看得见

27

条原始信号

用户反馈 CSV + 真实 GitHub Issue 多源导入

5

条可行动信号

情绪 / 误用 / 信息不足 → 滤出 22 条

3

个问题簇

1 簇经历史需求查重 → DUPLICATE 合并

1

个焦点需求

低优先级簇 → 分流 Later

P0

立即启动 • Now

机会总分 86.6

## 漏斗出口

22 条反馈滤出

1 簇查重合并

2 簇分流降级

演示数据刻意埋入「会被淘汰 / 分流」的样本——漏斗的价值正在于它敢于说「不做」。  
EvoPM Agent

# 需求初评 61 分没过线，自动补全证据后跃升 86



代码规则判定门禁（总分  $\geq 70$  且无 blocker），不是让 LLM 自己说「过」。FAIL 时系统用竞品 / 技术调研的真实证据自动补全一轮再重评——人工介入前先自我纠偏。



决策工作台 · 焦点需求质量雷达 (10 维)

10 维评分

锚点描述

blocker 维度

回炉计数限 1 次

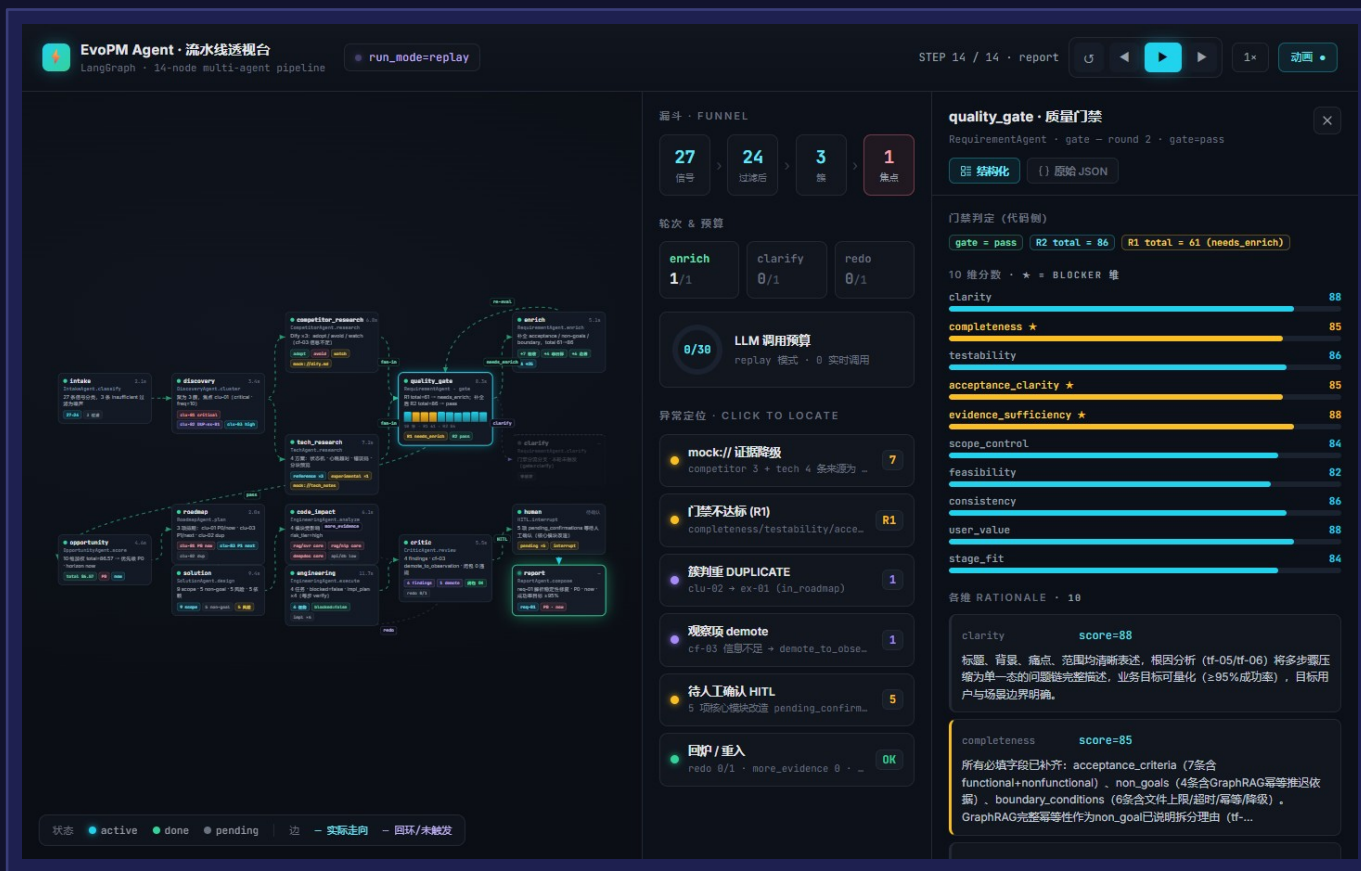
# 给评委与团队看「结论」的成品界面



浅色 Linear 风 · 6 屏左导航 · 接 glm-5.1 真实 replay 数据

- **焦点需求 HERO** 10 维质量雷达 + 总分 61→86 滚动, 初评 / 终评切换
- **可点击证据弹层** 原文摘录 + 来源 + 强度, mock:// 标本地材料
- **机会评分 · 路线图** Now / Next / Later 分流, DUPLICATE 标记
- **研发执行** 核心模块 ⚠ 高风险 + 任务卡 + 风险清单
- **报告中心** 4 份精排报告, [未确认] 结论标注

# 现场演示多智能体执行过程并定位问题



## 14 节点实时点亮

每个节点显示 agent / 一行结论 / 耗时 / 迷你指标

## 条件分支与回环明示

并行 fan-in、enrich 回环、未触发的 ghost 路径 (紫虚线)

## 播放 + 计数侧栏

▶ / ⏸ / 单步 / 倍速, 漏斗 · rounds · LLM 预算环

## 节点检视抽屉

结构化字段表 ↔ 原始 JSON 一键切换, 异常可定位

深色科技风 · 14 节点蛇形 DAG 实时点亮 · 节点检视抽屉



# 复杂的 Agent 逻辑 + 现场可靠的稳定性兜底

## 对抗式审查 Critic

专设审查智能体：5 档证据强度、揪过度推断、弱证据降权为观察项、生成待确认清单；本次跑出 20 条审查发现。

## 证据闭包代码校验

evidence\_refs 贯穿全部 schema，每个节点输出后由代码校验引用合法性，LLM 编造的引用被剔除并记入 Critic 输入。

## 人工介入分级

低风险自动通过 / 中风险进待确认 / 高风险（命中核心模块）强制确认；3 个 interrupt 断点，系统永不改代码。

## 稳定性兜底

LLM 429 指数退避重试、结构化输出校验重试、磁盘缓存 + --replay 离线重放、外部依赖失败自动降级 mock。

# 可运行的 MVP，且能落到任意活跃产品

## 应用价值

- 把「读反馈 → 排优先级 → 写 PRD」从数天人工压缩到一次自动化运行，每条结论可追溯。
- 数据源可插拔：任意有用户反馈 + GitHub 仓库的活跃产品 / 开源项目都能接入（本 Demo 用 82.6k★ 的 RAGFlow）。
- 证据闭包 + 人工分级，让 AI 决策可审计、可问责——这正是产品团队敢用 Agent 的前提。

## 完成度 · 可运行证据

14

节点全链跑通

137

测试通过 (10 skip)

2

套 React 前端

4

份 Markdown 报告

0

次 LLM 调用 (replay)

100%

离线可复现

`evopm run --replay`：断网也能端到端完整演示——演示路径可靠 > 工程完备。

# 从 Demo 到能持续进化的产品决策中枢

## NOW

### 打磨 Demo

- 后端实时 API 串联两套前端
- 门禁 / 评分权重可视化调参
- 更多真实 Issue 样本回归

## NEXT

### 走向闭环

- 实时反馈流接入（工单 / 社区 / 应用商店）
- 决策落地后的追踪与回环验证
- 多产品 / 多模块并行分析

## LATER

### 生态与规模

- 深度集成 SoloVerse 生态工具
- 团队协作与评审工作台
- 决策知识库沉淀与复用



# 让每一个产品决策 都有证据 · 有出口 · 可追溯

---

基于 LangGraph 的多智能体产品需求决策系统 · SoloVerse · Agent 元宇宙黑客松

[github.com/Adversit/Product-Evolution-Agents](https://github.com/Adversit/Product-Evolution-Agents)

谢谢观看